

Campus Resource Management System

Technical Report

1 Class Design

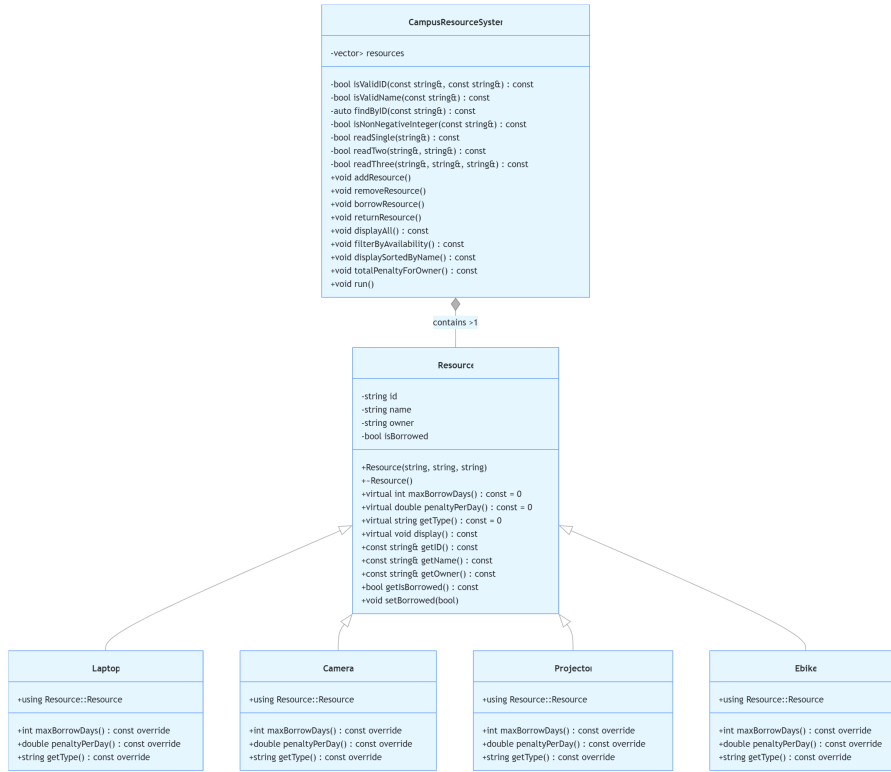


Figure 1: UML Class Diagram of Campus Resource System

The **Resource** class is abstract because it contains pure virtual methods (`maxBorrowDays()`, `penaltyPerDay()`, `getType()`) that have no implementation. This forces all derived resource types to implement these behaviours, ensuring a consistent interface while allowing type-specific logic.

2 C++ Features

(a) **Smart Pointers** The system uses `std::unique_ptr<Resource>` to manage dynamically allocated resources:

```
1 std::vector<std::unique_ptr<Resource>> resources;
```

```
1 std::unique_ptr<Resource> ptr;
```

```
2 if (type == "Laptop") ptr = std::make_unique<Laptop>(id, name, owner);
```

```
3 else if (type == "Camera") ptr = std::make_unique<Camera>(id, name, owner);
```

```

4 else if (type == "Projector") ptr = std::make_unique<Projector>(id, name,
    owner);
5 else ptr = std::make_unique<Ebike>(id, name, owner);

```

This eliminates raw pointers and manual `new/delete`, preventing memory leaks and simplifying ownership semantics.

(b) **Lambda Expressions** A lambda is used in the `findByID` helper function:

```

1 auto findByID(const std::string& id) const {
2     return std::find_if(resources.begin(), resources.end(),
3         [&id](const auto& r) { return r->getID() == id; });
4 }

```

The lambda captures the `id` by reference and acts as a predicate for the STL algorithm, keeping the code concise and readable.

(c) **STL Algorithm** `std::sort` is used to display resources sorted by name:

```

1 std::vector<const Resource*> tmp;
2 for (const auto& r : resources) tmp.push_back(r.get());
3
4 std::sort(tmp.begin(), tmp.end(), [](const Resource* a, const Resource* b) {
5     return a->getName() < b->getName();
6 });
7
8 for (const auto& r : tmp) r->display();

```

This leverages the STL to perform efficient, idiomatic sorting instead of writing a custom algorithm.

(d) **Move Semantics** When adding resources, `std::move` transfers ownership of the smart pointer:

```

1 resources.push_back(std::move(ptr));

```

This avoids expensive copies of `unique_ptr` and ensures efficient transfer of ownership to the vector.

3 Testing

Input	Expected Output	Actual Output	Result
Add: Laptop, L001, ThinkPad, CS-Dept	Resource L001 added.	Resource L001 added.	Pass
Add: Laptop, L9999, MacBook, IT	Invalid input. Try again.	Invalid input. Try again.	Pass
Add: Laptop, L001, MacBook, IT	Invalid input. Try again.	Invalid input. Try again.	Pass
Return: C001, 0	No penalty.	No penalty.	Pass
Menu choice: 10	Invalid input. Try again.	Invalid input. Try again.	Pass

Table 1: Test cases covering valid/invalid input scenarios

All tests passed as expected, confirming correct validation logic and output behaviour.

4 Reflection

One concrete improvement is to add **file persistence** to save and load resources from a text file between runs. Currently, all data is stored in memory and lost when the program exits. By implementing a simple text-based save/load mechanism, users could manage resources across multiple sessions without re-entering data each time. This would make the system more practical for real-world use and align with common resource management workflows.